

Métrologie, Prometheus et Grafana

Cours 1 - déployer et comprendre une stack de monitoring
Kubernetes avec des manifests simples

Prometheus

Grafana

Kubernetes manifests

Sans Helm en TP

Collecter

scrape /metrics

Analyser

PromQL

Visualiser

dashboards

Alerter

Alertmanager

À la fin du cours, un développeur sait rendre son service observable.

Le but n'est pas de devenir ops : c'est de savoir mesurer, lire et défendre le comportement d'un microservice.

- Expliquer le rôle des métriques dans l'observabilité.
- Déployer Prometheus, Alertmanager, exporters et Grafana en YAML.
- Instrumenter une application via un endpoint /metrics.
- Construire un dashboard utile pour diagnostiquer.

POSTURE

Observer avant de deviner

Un incident se traite avec des signaux, pas seulement avec une intuition.

LIVRABLE

Rendre les sources

Les manifests et le dashboard deviennent les preuves de compréhension.

Métriques, logs et traces ne répondent pas aux mêmes questions.

Prometheus et Grafana couvrent surtout les métriques ; ELK prendra le relais pour les logs au cours 2.

Métriques

Combien ? À quelle vitesse ?

requêtes/s, latence p95, CPU, pods disponibles

Logs

Que s'est-il passé ?

stack trace, utilisateur, payload refusé, message métier

Traces

Quel chemin ?

API A → service B → base C, durée par étape

Un dashboard répond à une question. Il ne doit pas être une galerie de graphes.

Kubernetes transforme un conteneur en service répliquable et adressable.

Pour monitorer proprement, il faut suivre les objets stables plutôt que les IP éphémères.



Dans le TP, Prometheus ne surveille pas une IP de Pod : il découvre des objets Kubernetes et suit les labels/annotations.

ConfigMap et RBAC font partie du monitoring, pas des détails administratifs.

Prometheus et kube-state-metrics doivent lire l'état du cluster pour découvrir et décrire les cibles.

- ConfigMap : prometheus.yml, règles d'alerte, datasource Grafana.
- ServiceAccount : identité portée par un Pod.
- ClusterRole : droits de lecture sur pods, services, endpoints, nodes.
- ClusterRoleBinding : rattache les droits à l'identité.

POINT DE CONTRÔLE

Si kube-state-metrics échoue

Commencer par vérifier le ServiceAccount, le ClusterRoleBinding et les logs du pod.

Une bonne stack mesure le noeud, le cluster, le code et le métier.

Chaque famille répond à une question différente ; les mélanger sans intention crée du bruit.

SYSTÈME

Noeud

CPU, mémoire, disque, réseau. Source principale : node-exporter.

CLUSTER

Kubernetes

Pods, Deployments, Nodes, Jobs. Source principale : kube-state-metrics.

APPLICATION

Code

Requêtes, erreurs, latence, files, dépendances. Source : /metrics.

MÉTIER

Usage

Commandes, paiements, imports, documents, parcours fonctionnels.

Une stack utile couvre ces quatre niveaux, mais le développeur doit surtout maîtriser les métriques applicatives et métier.

Prometheus stocke des séries temporelles numériques, pas des événements.

Une métrique est un nom, des labels et des valeurs horodatées.



```
http_requests_total{method="GET",code="200",service="demo-app"} 1234
```

Counter, gauge et histogram : choisir le type avant de coder.

Le type de métrique conditionne les requêtes PromQL et la qualité du dashboard.

AUGMENTE

Counter

Nombre total de requêtes, erreurs ou traitements. À lire avec `rate()` ou `increase()`.

```
rate(...[5m])
```

VARIE

Gauge

Valeur qui monte et descend : mémoire, connexions, pods disponibles.

```
kube_pod_status_phase
```

DISTRIBUE

Histogram

Latence ou taille de réponse par buckets. Permet les percentiles.

```
histogram_quantile()
```

À DISCUTER

Summary

Quantiles côté application, moins pratique à agréger entre réplicas.

```
import_duration_seconds{quantile="0.9"}
```


RED donne le premier dashboard utile d'un service HTTP.

Rate, Errors, Duration : trois questions pour savoir si l'utilisateur souffre.

- Rate : combien de requêtes arrivent ?
- Errors : quelle proportion échoue ?
- Duration : combien de temps les requêtes prennent-elles ?
- C'est le socle applicatif demandé dans le TP.

RATE

```
sum(rate(http_requests_total[1m]))
```

ERRORS

```
rate({code=~"4..|5.."}[5m]) / rate(total[5m])
```

DURATION

```
histogram_quantile(0.95, ...bucket[5m])
```

USE et SLO relient les ressources à un objectif de service.

Une ressource saturée n'est intéressante que si elle menace le service rendu.

- USE : Utilization, Saturation, Errors.
- SLI : indicateur mesurable de niveau de service.
- SLO : objectif explicite sur une période.
- Exemple : 99,5 % des requêtes réussies sur 30 jours.

RESSOURCE

CPU à 95 %

Signal utile si la latence ou les erreurs augmentent.

SERVICE

p95 > 500 ms

Signal directement lié à l'expérience utilisateur.

Prometheus est adapté aux microservices dynamiques.

Son modèle pull, ses labels et PromQL sont particulièrement pratiques dans Kubernetes.

- Chaque cible expose ses métriques en HTTP.
- Prometheus détecte aussi les cibles down.
- Les labels permettent de filtrer par service, namespace, pod.
- PromQL transforme les compteurs en signaux temporels.

FORCE

Multi-dimensionnel

Même métrique, plusieurs labels : service, code, méthode, instance.

LIMITE

Pas pour tout stocker

Les détails unitaires appartiennent aux logs, pas aux labels Prometheus.

La découverte Kubernetes remplace la liste statique de Pods.

Le TP utilise les annotations pour décider ce que Prometheus doit scraper.

```
metadata:  
  annotations:  
    prometheus.io/scrape: "true"  
    prometheus.io/path: /metrics  
    prometheus.io/port: "8080"
```

1. Kubernetes SD

Prometheus observe les pods, services et endpoints.

2. Relabel keep

On garde seulement les objets annotés scrape=true.

3. Target final

Prometheus reconstruit l'adresse host:port à scraper.

Trois sources couvrent noeud, cluster et code applicatif.

Le dashboard devient lisible quand chaque métrique a une provenance claire.

- node-exporter : métriques Linux de chaque noeud.
- kube-state-metrics : état des objets Kubernetes.
- demo-app : métriques custom exposées par le service.

NOEUD

node_cpu_seconds_total

CLUSTER

kube_deployment_status_replicas_available

APP

http_requests_total

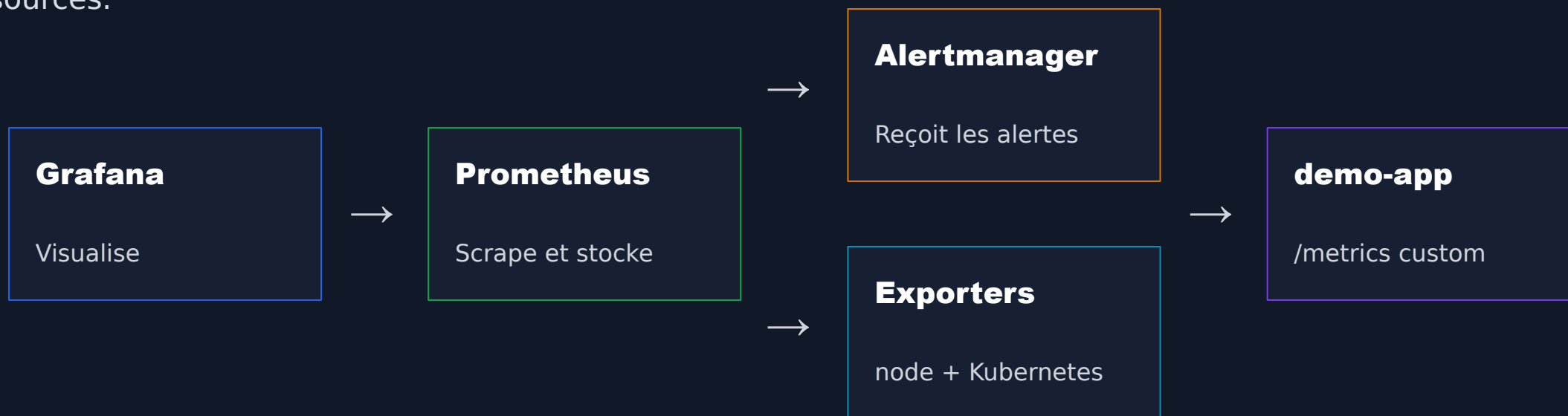
Le choix de déploiement change ce que les étudiants apprennent.

Méthode	Ce que l'on gagne	Ce que l'on perd	Bon usage
Manifests	Objets visibles, corrigibles	Verbeux, maintenance manuelle	TP et compréhension
Helm	Rapide, standard, configurable	Cache beaucoup de YAML	Équipe qui maîtrise les charts
Operator	CRD, automatisation, prod	Plus abstrait, plus complexe	Plateforme Kubernetes mature
kube-prom-stack	Stack complète + dashboards	Trop magique pour débiter	Bootstrap avancé
Managé	Moins d'exploitation	Coût, dépendance fournisseur	Entreprise / cloud

Le TP sera fait avec manifests manuels pour rendre les objets vérifiables, mais Helm reste un standard à connaître.

Le TP montre la stack complète, sans Operator.

Les étudiants rendent leurs manifests pour que la correction valide les sources.



Tout est déployé en YAML : aucun chart Helm, aucune CRD Operator, aucun ServiceMonitor.

Chaque fichier YAML porte une responsabilité identifiable.

Cette granularité rend le rendu étudiant lisible et testable sur un cluster vierge.

00-namespaces.yaml

Sépare monitoring et demo.

02-prometheus-config.yaml

Scrape configs + règles d'alerte.

05-node-exporter.yaml

DaemonSet : un exporter par noeud.

08-grafana-deployment-service.yaml

UI + datasource Prometheus.

01-prometheus-rbac.yaml

Autorise la découverte Kubernetes.

04-alertmanager.yaml

Réception et regroupement d'alertes.

06-kube-state-metrics.yaml

État Kubernetes transformé en métriques.

09-demo-app.yaml

Application instrumentée via /metrics.

PromQL transforme les compteurs bruts en signaux lisibles.

Les requêtes du TP restent simples, mais elles couvrent le diagnostic de base.

Targets disponibles

```
up
```

Trafic HTTP

```
sum(rate(http_requests_total[1m]))
```

Codes HTTP

```
sum by (code) (rate(http_requests_total[1m]))
```

Latence p95

```
histogram_quantile(0.95, sum(rate(http_request_duration_seconds_bucket[5m])) by (le))
```

CPU noeud

```
100 * (1 - avg by (instance) (rate(node_cpu_seconds_total{mode="idle"}[5m])))
```

Grafana visualise une question, pas juste une requête.

Une datasource Prometheus est provisionnée par ConfigMap ; le dashboard reste éditable/exportable.

Trafic applicatif

req/s



tendance 5m



Erreurs

4xx/5xx



budget



Latence p95

p95



p50



Cluster

Pods running



CPU noeud



Le dashboard demandé suit le diagnostic développeur.

Il doit montrer le comportement de l'application, puis le contexte Kubernetes.

- Trafic HTTP : requêtes par seconde.
- Codes HTTP : succès et erreurs.
- Taux d'erreur : proportion exploitable.
- Latence p95 : perception utilisateur.
- Pods et CPU : contexte de déploiement.

QUALITÉ

Un bon panel a une unité

Seconds, percent, req/s, bytes : l'unité évite les contresens.

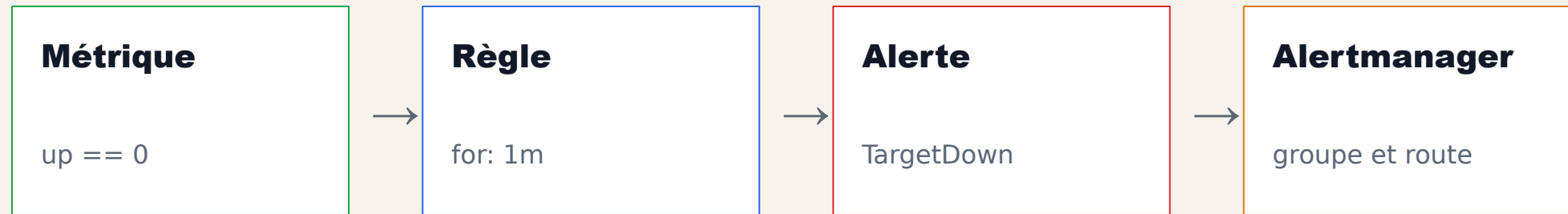
LISIBILITÉ

Moins de séries, plus de signal

Direct labels et panels ciblés valent mieux qu'un dashboard encyclopédique.

Alertmanager commence là où Prometheus décide qu'il y a un problème.

Pour le TP, l'objectif est de voir une alerte arriver, pas de configurer une astreinte réelle.



```
expr: up == 0  
for: 1m  
labels: { severity: warning }
```

La validation se fait sur un cluster vierge.

Le rendu doit prouver qu'il est reproductible et non dépendant d'une configuration manuelle dans l'UI.

kubectl apply

Les manifests s'appliquent sans Helm.

Targets Prometheus

Prometheus, exporters et app sont up.

Datasource Grafana

Prometheus est provisionné.

Dashboard

Panels lisibles après trafic.

Alertmanager

Une alerte est visible.

README

Commandes et choix expliqués.

Les erreurs de monitoring se trouvent presque toujours dans trois zones.

On diagnostique la chaîne de collecte comme une application distribuée.

- Discovery : annotations absentes ou port incorrect.
- Réseau : Service ou endpoint non joignable.
- Permissions : RBAC insuffisant pour lire l'API Kubernetes.
- Grafana : datasource mal pointée ou ConfigMap non rechargée.

PROMETHEUS

Status > Target health

KUBERNETES

kubectl logs / describe

GRAFANA

Data sources > Prometheus

Le rendu évalue la compréhension, pas l'installation.

Les sources doivent permettre une correction automatique ou semi-automatique.

```
nom-prenom-prometheus-grafana/  
  README.md  
  manifests/  
    00-namespace.yaml  
    ...  
  grafana/  
    dashboard.json
```

- Manifests lisibles, non générés par Helm.
- Images versionnées et labels cohérents.
- Prometheus découvre les targets automatiquement.
- Dashboard exporté et expliqué.
- Alerte test documentée.

Prometheus détecte le signal ; ELK aidera à chercher l'événement exact.

Le cours 2 partira des logs : index, recherche, dashboards Kibana et corrélation avec les métriques.

Prometheus détecte

Hausse du taux d'erreur, latence p95, cible down, saturation CPU.



ELK explique

Stack trace, payload, utilisateur, corrélation temporelle, recherche plein texte.

Cours 2 : partir d'un signal métrique et chercher l'événement exact dans les logs.

L'observabilité se conçoit dans le service.

Le monitoring n'est pas une couche magique ajoutée à la fin.

01 Une métrique doit aider à décider.

02 La cardinalité des labels est un choix de conception.

**03 node-exporter mesure les noeuds ;
kube-state-metrics décrit Kubernetes.**

**04 Prometheus collecte et alerte ;
Grafana visualise.**

**05 Helm est utile, mais les manifests
montrent ce qu'on comprend.**

**06 Les logs complètent les métriques
quand il faut expliquer le pourquoi.**